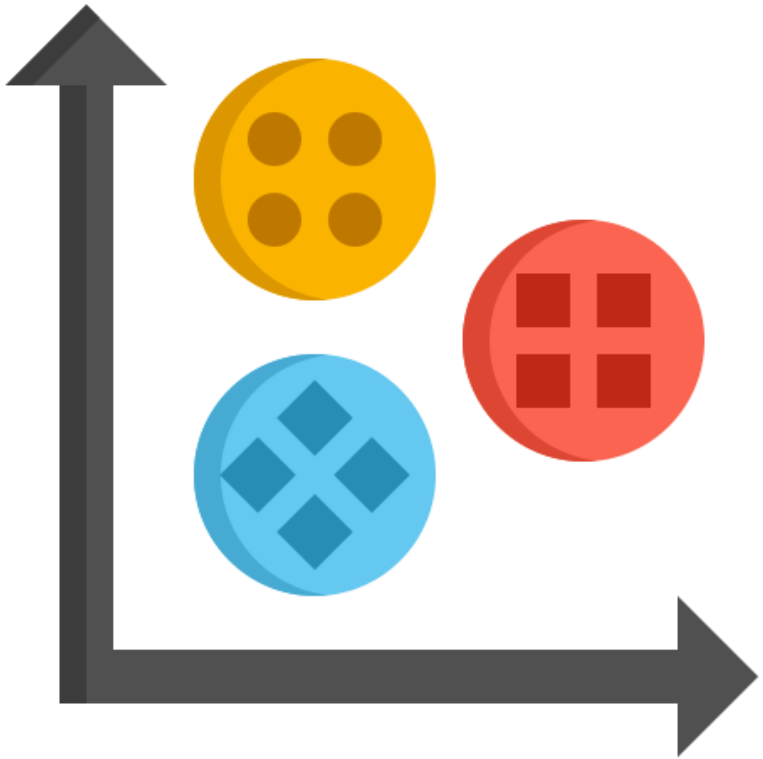
Clustering



Outline

- Introduction to Clustering
- K-Means Clustering
- DBSCAN
- Clustering Evaluation Metrics

Clustering

Clustering is an **unsupervised learning** technique that groups data points without labels using two main approaches:

- **Similarity-Based Clustering**: Groups points based on distance or similarity in feature space
  - Uses distance metrics like Euclidean, Manhattan, Cosine
  - Maximize **intra-cluster** similarity (compactness) and maximize **inter-cluster** separation
  - Assumes clusters are compact and often spherical
  - Examples: **K-Means**, Hierarchical Clustering
- **Density-Based Clustering**: Detects clusters as dense regions separated by low-density gaps
  - Captures arbitrary shapes (e.g., snakes, crescents)
  - Handles noise/outliers
  - Examples: **DBSCAN**, HDBSCAN

Applications

🎯 **Customer Segmentation:** Group customers with similar demographics or purchasing behavior to enable targeted marketing and personalized offers.

🛒 **Product Recommendation:** Cluster users or items based on interaction patterns to improve recommendation quality.

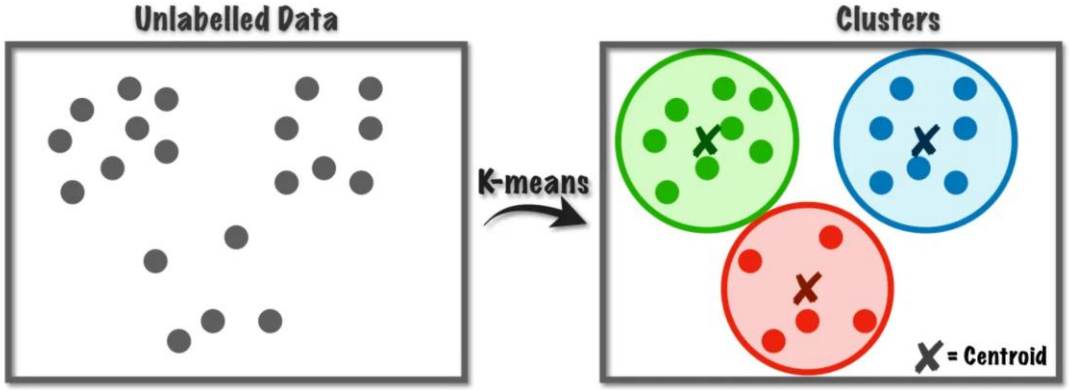
🗂️ **Document & Text Topic Grouping:** Cluster articles, reviews, or documents by similarity to organize large text collections.

🩺 **Medical & Health Analytics:** Discover patient subgroups with shared symptoms or risk factors for improved diagnostics.

🔍 **Anomaly & Intrusion Detection:** Detect unusual or suspicious activity—such as abnormal network traffic—by detecting points that fall outside all clusters.

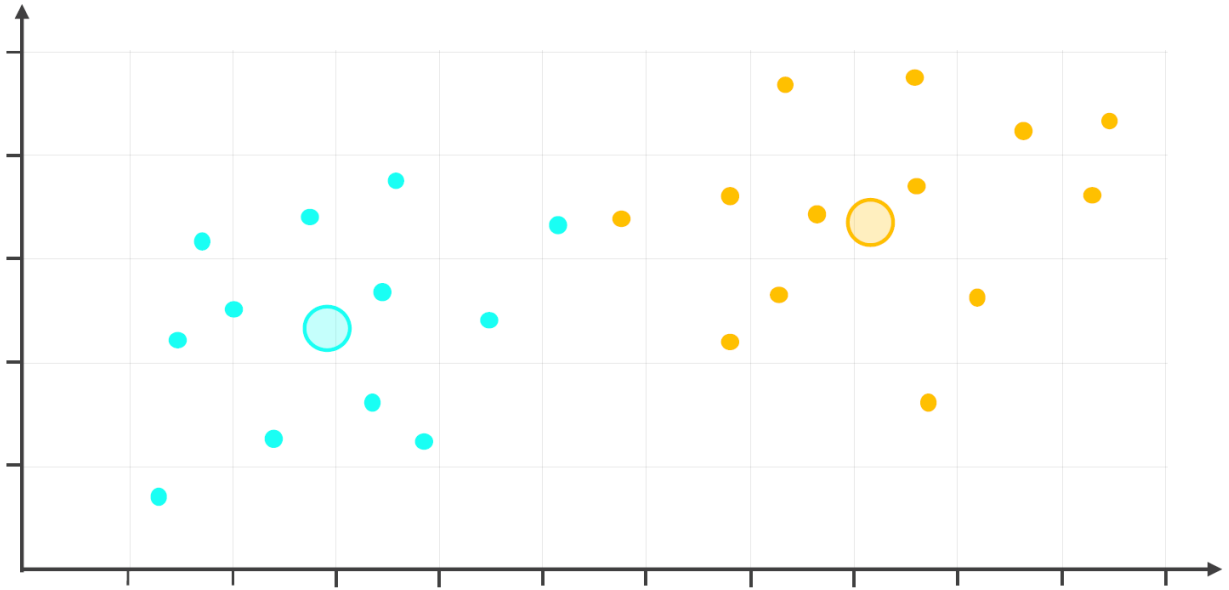
🤖 **ML Preprocessing & Data Reduction:** Simplify datasets by grouping similar samples or features to support efficient analysis and modeling.

K-Means Clustering



K-Means Clustering

- K-Means is a clustering algorithm that **partitions** data into K groups by assigning each observation to the cluster with the **nearest centroid**, where K is the number of clusters defined by the user



K-Means Algorithm

How it Works (Iterative Optimization)

1. **Initialization**: Select K random points as initial centroids (typically using **k-means++** to spread them out)
2. **Assignment Step**: Assign each data point to its closest centroid based on Euclidean distance
3. **Update Step**: Recalculate centroids by taking the mean of all points assigned to that cluster
4. **Convergence**: Repeat until centroids do not change or change is below a tolerance threshold

Mathematical Objective

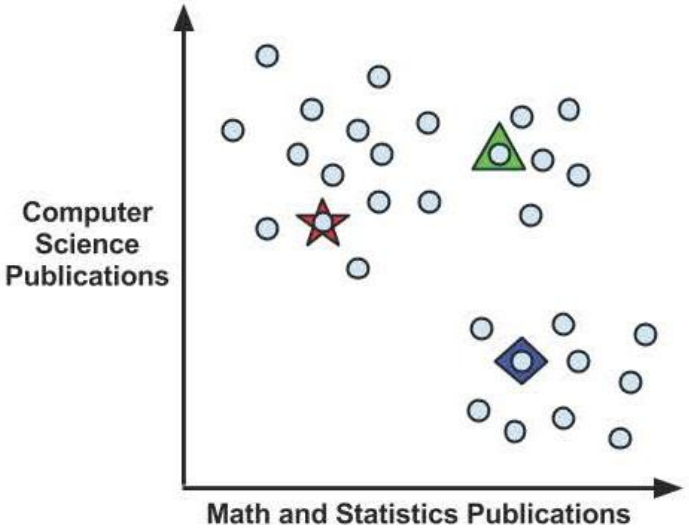
- Minimizes **Inertia** (Within-Cluster Sum of Squares - WCSS):
- $\sum_{i=0}^{n} \min_{\mu_j \in C} (||x_i - \mu_j||^2)$
- **Intuition**: It tries to make clusters as compact (spherical) as possible

Usage Scenarios

- **When to use**: You know K, clusters are roughly spherical, equal variance, and similar size.
- **Examples**: Market Segmentation, Document categorization.

Using distance to assign and update clusters

- k-means treats feature values as coordinates in a multidimensional feature space.
- It starts by selecting k initial centroids—typically chosen randomly from the dataset
- Each point is then assigned to its nearest centroid, after which the centroids are updated as the mean of their assigned points.
- This iterative process continues until the cluster assignments stabilize.

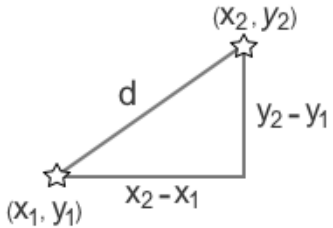


Euclidean distance

- Euclidean distance measures how far two points are in a geometric space
- For two points: $x = (x_1, x_2, . . . x_n), y = (y_1, y_2, . . . y_n)$

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2}$$

- k-means relies on Euclidean distance to decide **cluster membership** and **update centroids**



$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K-Means: Pros & Cons ⚖️

✅ **Benefits**

- **Efficiency**: Very fast, generally ($O(n)$) - Linear scaling with n
- **Easy Implementation**: Standard baseline for clustering
- **Scalable**: Works well on large datasets
- **Interpretable**: Centroids represent the "average" of the cluster

⚠️ **Limitations**

- **Need to choose K**: The number of clusters must be specified upfront
- **Sensitive to Initialization**: Bad starting points can lead to poor results (mitigated by `k-means++`)
- **Spherical Assumption**: Fails on irregular shapes (e.g., crescents) or clusters with different densities/sizes
- **Sensitive to Outliers**: Outliers can pull centroids away from the true center
- **Curse of Dimensionality**: In high dimensions, Euclidean distance becomes less discriminative (distances converge)

K-Means Clustering using scikit-learn

- You can use sklearn's **KMeans()** function to perform K-Means Clustering

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, n_init='auto', random_state=42)
```

The "k" number of clusters to identify (default is 8)

The number of models to fit with different initial centroids, returning the best result ("auto" will fit one model)

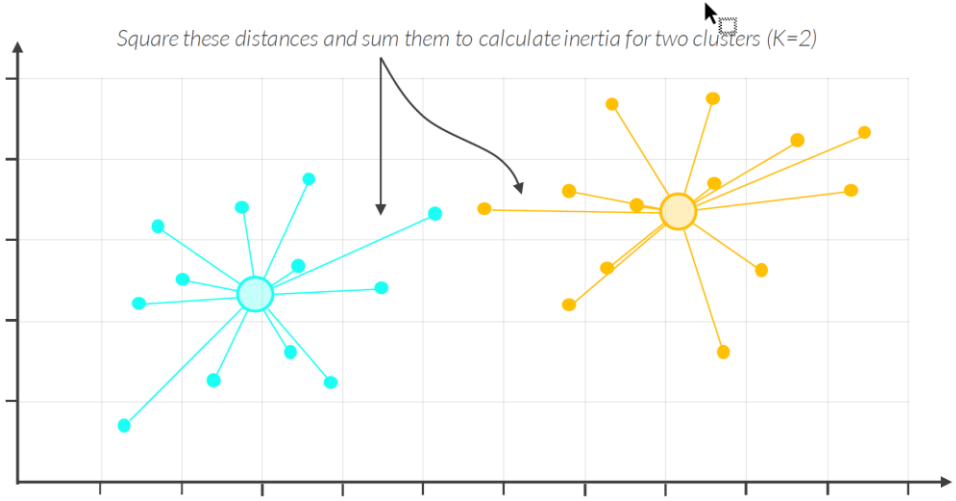
Setting a random_state value guarantees the same results each time the model is fit

K-Means Implementation 💻

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate synthetic data with distinct blobs
X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)

# Implementation
# init='k-means++': Smarter initialization to speed up convergence
# n_init=10: Run algorithm 10 times with different seeds, pick best result (lowest inertia)
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=10, random_state=42)
y_pred = kmeans.fit_predict(X)

# Visualization
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_pred, s=50, cmap='viridis', edgecolors='k')
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.9, marker='X', label='Centroids')
plt.title("K-Means Partitioning (Voronoi Cells)")
plt.legend()
plt.show()
```

Choosing K using Inertia

- Inertia, also called the Within-Cluster Sum of Squares (WCSS), measures how internally coherent each cluster is
- It represents the total squared distance between every point and the centroid of the cluster it belongs to
  - Lower inertia ⇒ more compact clusters
- If we have **K** clusters, each with centroid μ_j ,and data points x_i ,then:



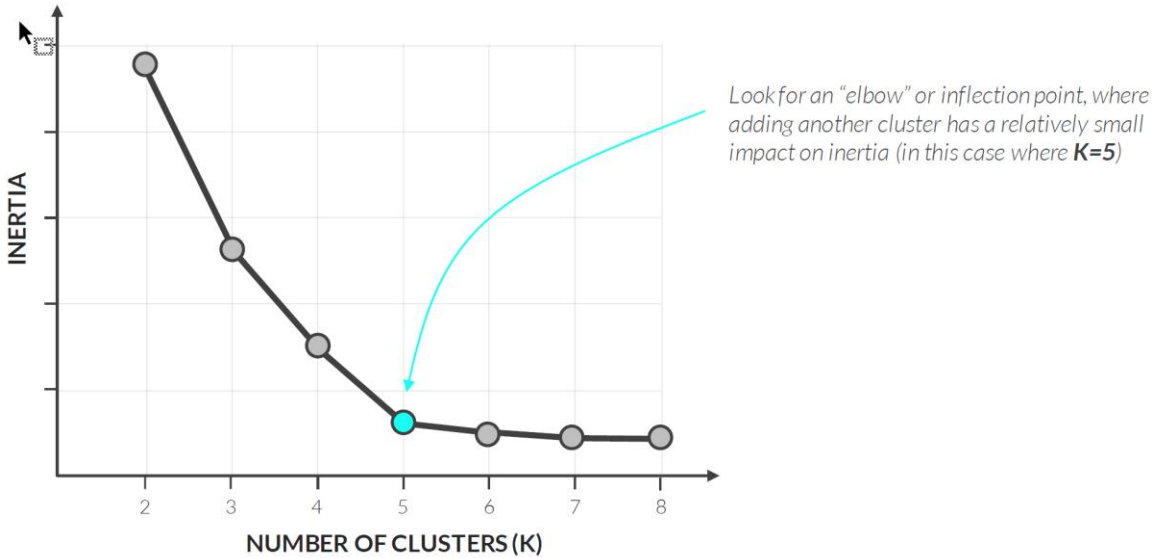
$$\text{Inertia} = \sum_{j=1}^{K} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

Where:

- C_j = the set of points assigned to cluster j
- μ_j = centroid of cluster j
- $\|x_i - \mu_j\|^2$ = squared Euclidean distance between a point and its centroid

The Elbow Method

- Run K-Means with several different values of K, compute the inertia (WCSS) for each model, and plot inertia versus K
- The “elbow” point—where the rate of decrease sharply slows—indicates a good balance between model simplicity and cluster compactness.

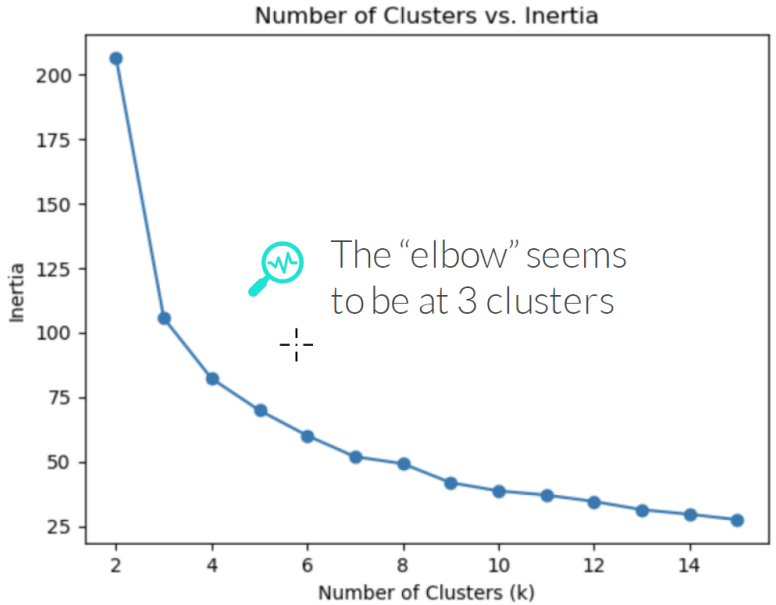


Choosing K: The Elbow Method 📐

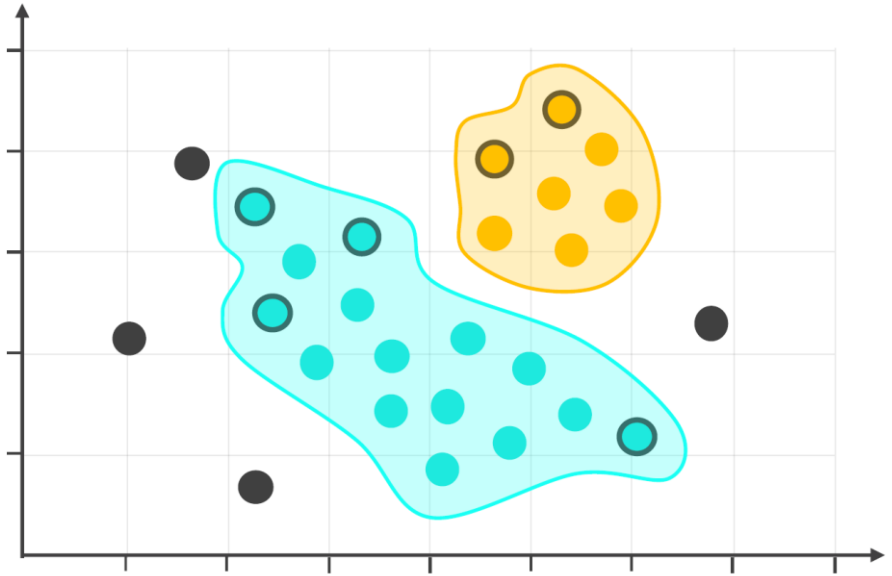
```python
# fit k-means models for 2-15 clusters, note the intertia scores
inertia_values = []

for k in range(2, 16):
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=30)
    kmeans.fit(data)
    inertia_values.append(kmeans.inertia_)
```

Fit models using 2 to 15 clusters and append their inertia values to a list

```python
# plot the inertia values
import matplotlib.pyplot as plt

# turn the list into a series for plotting
inertia_series = pd.Series(inertia_values, index=range(2, 16))

# plot the data
inertia_series.plot(marker='o')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.title("Number of Clusters vs. Inertia");
```

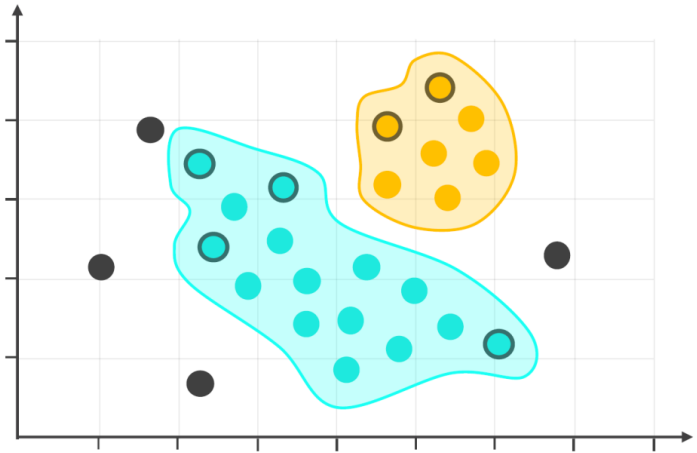


DBSCAN



DBSCAN

- DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering technique that identifies clusters based on the density of data points
  - Crucial distinction: K-Means partitions *the entire space into clusters*. DBSCAN Leaves sparse regions unassigned (noise), producing more realistic cluster boundaries



DBSCAN Algorithm

How DBSCAN Works (Density Reachability)

DBSCAN forms clusters as **continuous high-density regions** separated by low-density gaps. It requires two parameters:

- **`eps` (ϵ)**: The **maximum distance** between two points for them to be considered neighbors.
- **`min_samples`**: The minimum number of points required within a point's ε radius to classify it as a core point

Point Types

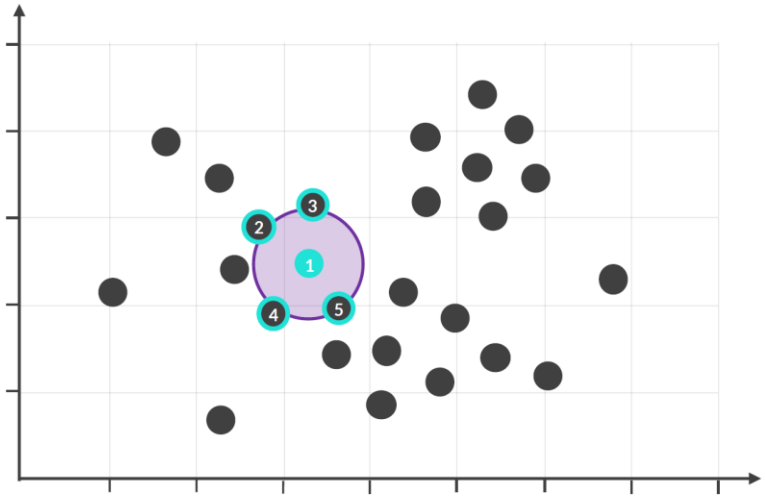
- **Core Point**: Has ≥ `min_samples` within `eps` radius (Interior of cluster)
- **Border Point**: Within `eps` of a Core Point but has < `min_samples` (Edge of cluster)
- **Noise Point**: Neither Core nor Border (Outlier)

Key Intuition

- **"Friends of Friends"**: If A is close to B, and B is close to C, DBSCAN links them into the same cluster when density is sufficient
- **Shape-Flexible:** Finds clusters of arbitrary shapes (crescents, spirals, nested structures)
- **Noise-Aware**: Points in sparse regions are labeled as noise instead of being forced into clusters

DBSCAN – Step 1

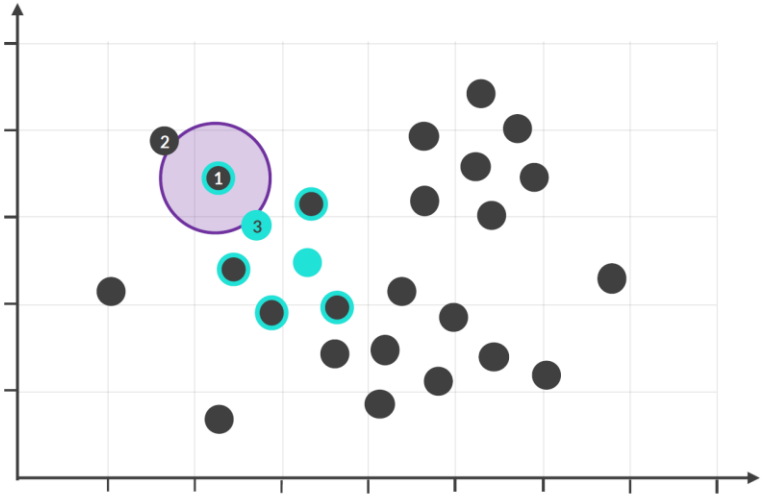
- STEP 1: Select a point at random and count the points within ϵ radius
  - If it's greater than or equal to the "min_samples", then start a cluster, label the point as a **core point**, and mark the points within its radius as a neighbor



eps = **0.75**
min_samples = **4**

DBSCAN – Step 2

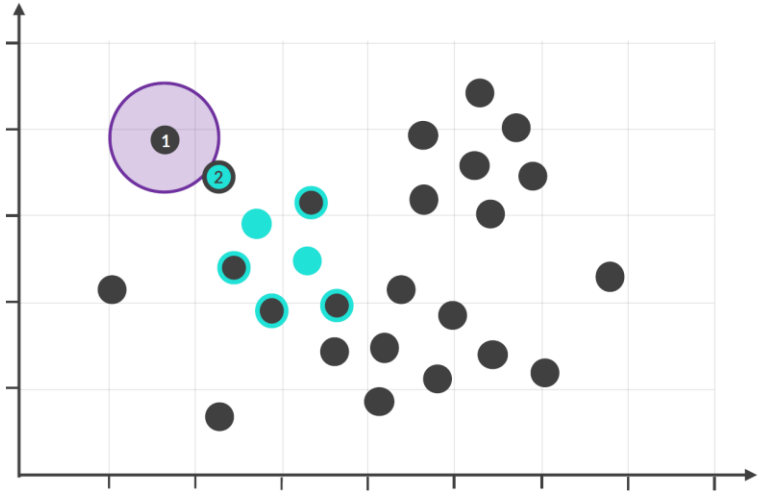
- **STEP 2:** Move to a neighbor and count the points within ϵ radius
  - If it's greater than or equal to the "min_samples", label it as a *core point*, and mark its *neighbors*
  - If it's less than "min_samples", but at least one of them is a core point, label it as a border point



eps = **0.75**
min_samples = **4**

DBSCAN – Step 2

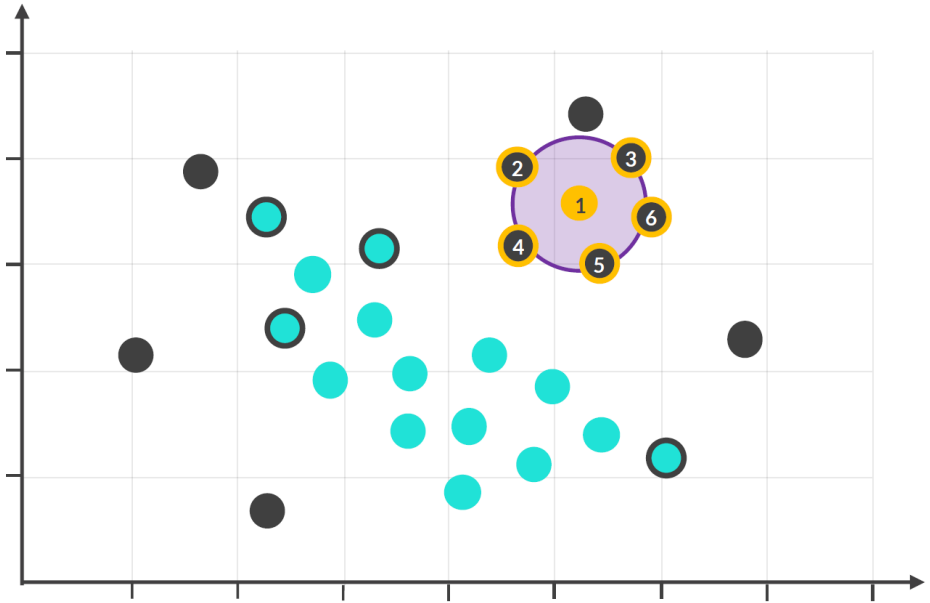
- **STEP 2:** Move to a neighbor and count the points within ϵ radius
  - If it's less than "min_samples", and none of them is a core point, label it as a noise point
- Move on to another neighbor and continue this process until all the points are labeled within the cluster



eps = **0.75**
min_samples = **4**

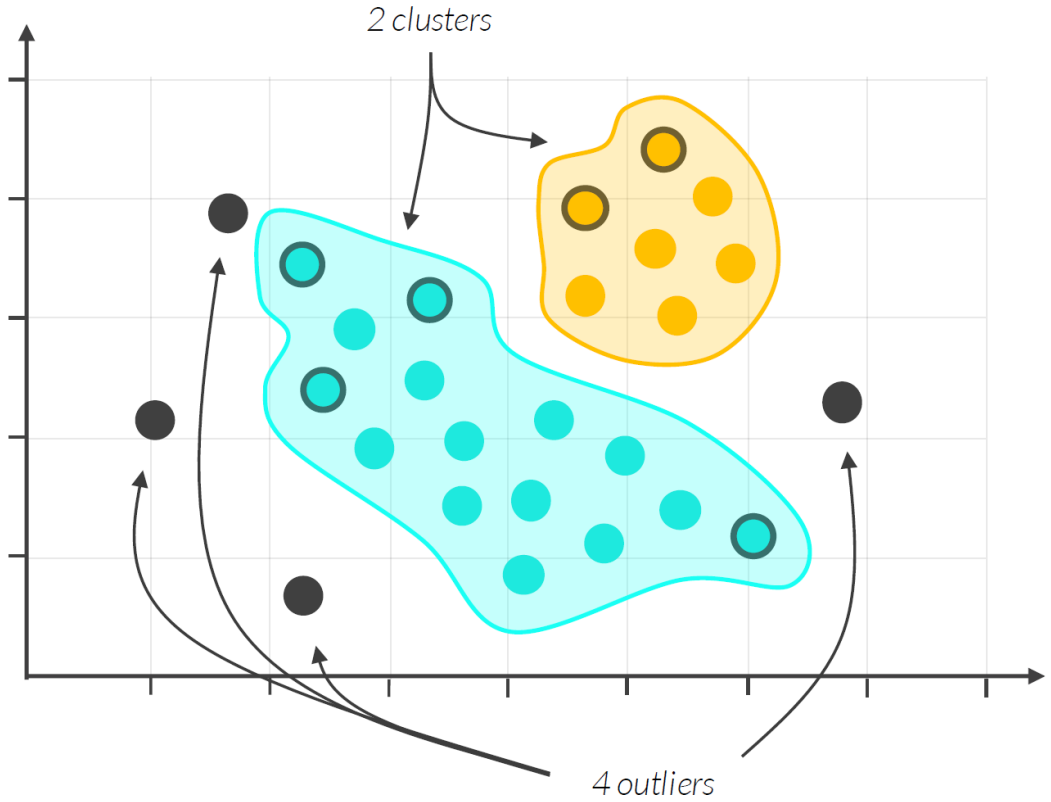
DBSCAN – Step 3

- **STEP 3:** Move on to another random point and repeat the same steps



eps = **0.75**
min_samples = **4**

Final DBSCAN Clusters



eps = **0.75**
min_samples = **4**

Algorithm Steps

1. **Start with an unvisited point**: Pick an arbitrary point from the dataset that has not been visited yet.
2. **Find its neighbors**: Find all points within the ε radius of the chosen point.
3. **Classify the point**:
   - If the point has at least MinPts neighbors, it is classified as a **core point**, and a new cluster is started.
   - If the point has fewer than MinPts neighbors, it is temporarily marked as a noise point.
4. **Expand the cluster:**
   - If a core point is found, all of its neighbors are added to the new cluster.
   - This process is repeated for all new points added to the cluster. If any of them are also core points, their neighbors are also added, and so on.
5. This continues until the cluster cannot be expanded further.
6. **Repeat the process**: Once the cluster is fully expanded, the algorithm moves to a new, unvisited point and repeats the process from step 1.
7. **Finish**: The algorithm is complete when all points have been visited and assigned to a cluster or marked as noise.

DBSCAN SkLearn

- You can use sklearn's DBSCAN() to perform DBSCAN clustering in Python

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
```

The radius, or maximum distance between neighboring points (default is 0.5)

Minimum points in the radius needed to become a core point, including the point itself (default is 5)

DBSCAN SkLearn

```
# import dbscan from sklearn
from sklearn.cluster import DBSCAN
```

```
# fit a dbscan model
dbscan = DBSCAN(eps=0.5, min_samples=5)
dbscan.fit(data)
```

▾ DBSCAN

DBSCAN()

*You can view the cluster assignments using the **.labels_** attribute (-1 values represent noise points)*

```
# view the cluster assignments
dbscan.labels_
```

```
array([ 0, -1,  0,  1,  1,  1,  1,  1,  0,  1, -1,  0,  0,  0,  0, -1,  1,
        0,  1, -1,  1, -1,  1,  1,  1,  1,  0,  0,  0, -1,  1, -1,  1,  1,
        1,  1,  1, -1,  0,  1,  1,  1, -1,  0, -1,  1,  0, -1,  0, -1,  1,
        1, -1,  1,  1,  1,  0,  1, -1,  1,  0,  0,  0,  1,  1, -1,  0, -1,
       -1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1,  1,  1,  1,  0,  0,  0,
        1,  1,  0,  0,  1,  0,  1,  1,  1,  1,  1, -1,  1,  1,  0,  1,  0,
       -1,  1, -1,  1,  0,  1,  0, -1, -1, -1,  0,  0,  0,  1,  1,  0,  1,
        1,  1, -1, -1,  0,  0, -1, -1,  1,  1,  0,  0,  0,  1,  0,  1,  1,
        1,  1,  1,  1,  0,  0,  1, -1,  1,  1,  1, -1, -1,  0])
```

There are too many noise points (-1) here, let's tweak **eps** and **min_samples** to mainly see clusters instead

DBSCAN: Pros & Cons ⚖️

✅ Benefits

- **Shape Agnostic**: Detects clusters of arbitrary shapes (linear, nested, crescent-shaped)
- **Noise Handling**: Unlike K-Means, outliers do not distort the cluster (they are labeled as `-1`)
- **No K Required**: Number of clusters is determined by the data density
- **Few Parameters**: only `eps` and `min_samples` to tune

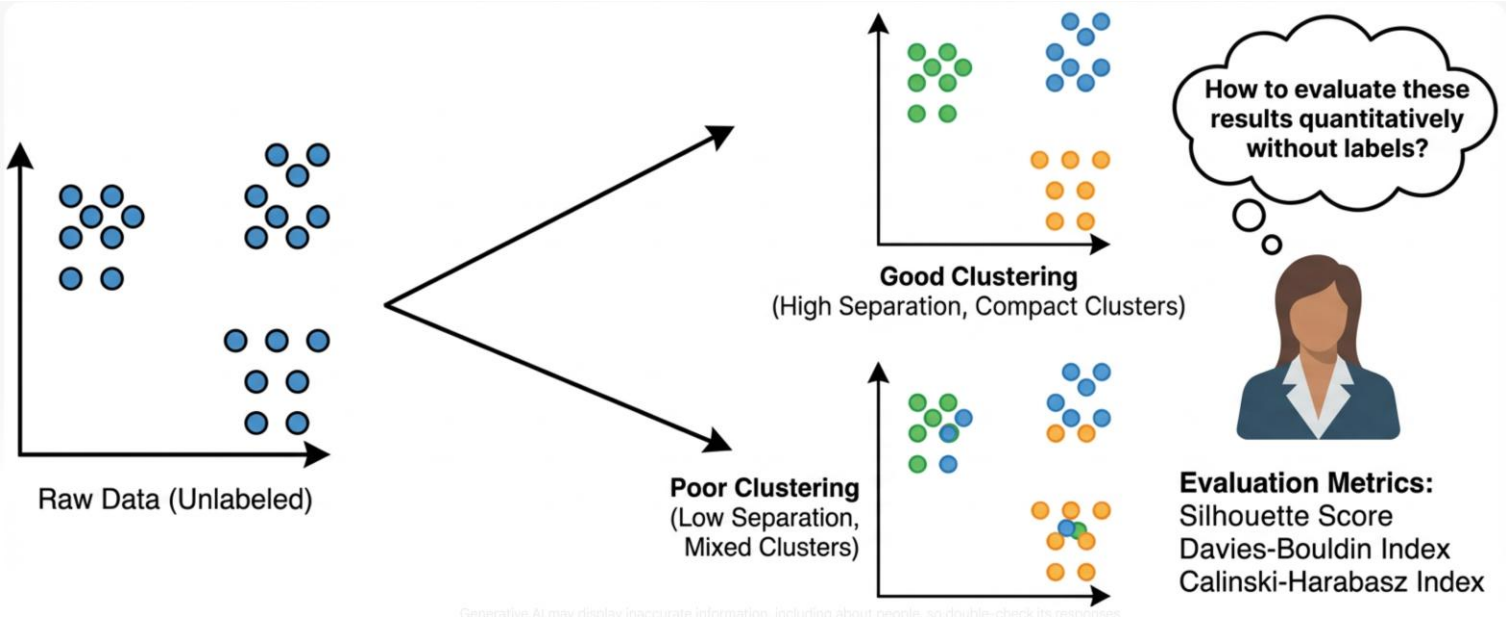
⚠️ Limitations

- **Varying Densities**: Performs poorly when clusters have significantly different densities (cannot find a single `eps` that works for all).
- **Parameter Sensitivity**: Finding the right `eps` can be tricky.
  - Small eps → many points become noise vs . Large eps → distinct clusters merge
- **High Dimensions**: "Curse of Dimensionality" implies Euclidean distance is less meaningful, making "radius" hard to define

DBSCAN Implementation & Tuning

```python
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# Generate non-spherical data (interleaved half-circles)
X, _ = make_moons(n_samples=300, noise=0.05, random_state=42)

# Implementation
# eps typically requires domain knowledge
dbscan = DBSCAN(eps=0.3, min_samples=5)
y_pred = dbscan.fit_predict(X)

# Visualization
plt.figure(figsize=(8, 6))
unique_labels = set(y_pred)
colors = [plt.cm.Spectral(each) for each in np.linspace(0, 1, len(unique_labels))]

for k, col in zip(unique_labels, colors):
    if k == -1:
        # Black used for noise.
        col = [0, 0, 0, 1]
        label = "Noise (Outliers)"
    else:
        label = f"Cluster {k} (Core+Border)"

    class_member_mask = (y_pred == k)
    xy = X[class_member_mask]

    # Plotting
    plt.plot(xy[:, 0], xy[:, 1], 'o', markerfacecolor=tuple(col),
             markeredgecolor='k', markersize=10 if k != -1 else 6, label=label)

plt.title("DBSCAN: Handling Arbitrary Shapes & Noise")
plt.legend()
plt.show()
```

Clustering Evaluation Metrics



Clustering Evaluation Metrics 📏

Why Evaluate Clustering? — the core challenge

- Clustering is **unsupervised** — no labels, no "correct answer"
- How do we know if clusters are *good*?
- Need **quantitative measures** to compare models objectively
  - **Silhouette Coefficient** — cohesion vs. separation
  - **Davies-Bouldin Index (DBI)** — average cluster similarity
  - **Calinski-Harabasz Score** — variance ratio criterion

📐 Key Concepts: Intra- vs. Inter-Cluster Distance

Building Blocks Shared by All Three Metrics

- **Intra-cluster distance (cohesion):** How *tight* are points within the same cluster?
  - 🎯 Smaller = better
- **Inter-cluster distance (separation):** How *far apart* are different clusters?
  - 🎯 Larger = better

- **The Golden Rule:** A good clustering has **compact** (tight) clusters that are **well-separated** from each other.
  - All three metrics operationalize this rule differently.

🔷 Silhouette Coefficient

Definition

The **Silhouette Coefficient** measures how similar a point is to its own cluster compared to neighboring clusters.

Range: **−1 to +1**

- **+1** → perfectly assigned (tight cluster, far from neighbors)
- **0** → on the boundary between two clusters
- **−1** → likely misclassified (closer to a neighboring cluster)

Silhouette — Step-by-Step Algorithm

How It Works

1. **Assign** each point to a cluster (run K-Means, DBSCAN, etc.)
2. **For each point** i:
   - Compute $\mathbf{a(i)}$: mean distance to all other points **in its cluster**
   - For each *other* cluster, compute mean distance from i to all points in that cluster
   - Set $b(i)$ = minimum of those means (**nearest other cluster**)
   - Compute $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$
3. **Average** $s(i)$ across all points → **overall Silhouette Score**

Complexity

- $O(n^2)$ distance computations — can be slow for large datasets

🔷 Silhouette — Intuition

Score Range	Interpretation
0.71 – 1.00	Strong clustering structure
0.51 – 0.70	Reasonable structure
0.26 – 0.50	Weak structure (may be artificial)
< 0.25	No substantial structure

🔷 Silhouette — When to Use

✅ Best Scenarios

- **Choosing k:** Run for $k = 2,3, \ldots, K$; pick the k with highest score
- **Comparing algorithms:** K-Means vs. Agglomerative — which fits the data better?
- **Diagnosing individual points:** `silhouette_samples` reveals poorly-assigned points

❌ Limitations

- **Assumes roughly spherical clusters** — poor for non- spherical shapes (use DBSCAN + DBI)
  - **Biased toward compact, globular clusters** — may prefer K-Means over DBSCAN
- $O(n^2)$ **cost** — slow for large datasets (> 50k points)
- **Ignores cluster size imbalance** — a tiny outlier cluster can inflate the score

🔷 Silhouette — Usage Example & Interpretation

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Generate synthetic data with 3 true clusters
X, y_true = make_blobs(n_samples=300, centers=3,
                       cluster_std=0.8, random_state=42)

# 2. Evaluate multiple k values
results = {}
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(X)
    score = silhouette_score(X, labels)
    results[k] = score
    print(f"  k={k} → Silhouette Score = {score:.4f}")

# 3. Pick best k
best_k = max(results, key=results.get)
print(f"\n✅ Best k = {best_k}  (score = {results[best_k]:.4f})")
```

```
  k=2 → Silhouette Score = 0.5879
  k=3 → Silhouette Score = 0.7302   ← best
  k=4 → Silhouette Score = 0.6114
  k=5 → Silhouette Score = 0.5423
  k=6 → Silhouette Score = 0.4801

✅ Best k = 3  (score = 0.7302)
```

Interpretation: Score of **0.73** → strong cluster structure. Scores decrease as k diverges from the true number of clusters

🔶 Davies-Bouldin Index (DBI)

Davies-Bouldin Index (DBI) measures the *average similarity* between each cluster and its most similar neighboring cluster.

Range: **0 to ∞**

- **Lower is better**
- **0** → perfect clustering (no overlap, maximally separated)
- Penalizes clusters that are **spread out** and **too close** to neighbors

Silhouette is per-point → aggregate; DBI is per-cluster → aggregate

DBI — Step-by-Step Algorithm

How It Works

1. **Compute centroids** μ_i for each cluster $i = 1 \ldots k$
2. **Compute scatter** s_i: average Euclidean distance from each point to its centroid
3. **For each pair** (i, j):
   - Compute inter-centroid distance $d(\mu_i, \mu_j)$
   - Compute $R_{ij} = (s_i + s_j)/d(\mu_i, \mu_j)$
4. **For each cluster** i: find $\max_{j \neq i} R_{ij}$ (worst neighbor similarity)
5. **Average** the worst-case values → **DBI score**

Complexity

- $O(n \cdot k + k^2)$ — much cheaper than Silhouette!
- Scales well to large datasets

DBI — Intuition

DBI Value	Interpretation
< 0.5	Very good separation
0.5 – 1.0	Acceptable clustering
> 1.0	Poor separation / overlapping clusters

🔶 DBI — When to Use

✅ Best Scenarios

- **Large datasets** where Silhouette's $O(n^2)$ cost is prohibitive
- **Centroid-based algorithms** (K-Means) where centroids are natural
- **Selecting k:** DBI typically dips at the optimal k
- **Quick sanity check** on model quality during hyperparameter tuning

❌ Limitations

- **Centroid-dependent** — less meaningful for DBSCAN or hierarchical clustering (no natural centroid for arbitrary-shaped clusters)
- **Assumes spherical clusters** — similar bias as Silhouette
- **Sensitive to outliers** — outliers inflate s_i (scatter), causing misleading scores

🔷 DBI — Usage Example & Interpretation

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import davies_bouldin_score

# 1. Generate data
X, _ = make_blobs(n_samples=300, centers=3,
                  cluster_std=0.8, random_state=42)

# 2. Evaluate k values — LOWER DBI is better
results = {}
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(X)
    score = davies_bouldin_score(X, labels)
    results[k] = score
    print(f"  k={k} → DBI = {score:.4f}")

best_k = min(results, key=results.get)    # ← NOTE: min for DBI
print(f"\n✅ Best k = {best_k}  (DBI = {results[best_k]:.4f})")
```

```
  k=2 → DBI = 0.7481
  k=3 → DBI = 0.3102    ← best (lowest)
  k=4 → DBI = 0.4856
  k=5 → DBI = 0.6213
  k=6 → DBI = 0.7894

✅ Best k = 3  (DBI = 0.3102)
```

Interpretation: DBI of **0.31** → excellent separation (well below 0.5 threshold)

Minimum correctly identifies $k=3$

Calinski-Harabasz Score (Variance Ratio Criterion)

Definition

The **Calinski-Harabasz Score** (CH) measures the ratio of between-cluster variance to within-cluster variance.

Also called the **Variance Ratio Criterion (VRC)**.

Range: **0 to ∞**

- **Higher is better**
- High score → clusters are **dense** and **well separated**

Calinski-Harabasz — Step-by-Step Algorithm

How It Works

1. **Compute grand mean** μ of all points
2. **Compute cluster centroids** μ_k for each cluster k
3. **Compute BCSS** (between-cluster scatter):
   - For each cluster: weight its centroid's distance from grand mean by cluster size
   - BCSS $= \sum_k n_k \parallel \mu_k - \mu \parallel^2$
4. **Compute WCSS** (within-cluster scatter):
   - For each point: squared distance to its cluster centroid
   - WCSS $= \sum_k \sum_{x \in C_k} \parallel x - \mu_k \parallel^2$
5. **Apply correction factor** and divide:
   - CH = (BCSS/WCSS) $\times (n - K)/(K - 1)$

Complexity

- $O(n \cdot d)$ — **linear** in n and feature dimension d — very fast!

Calinski-Harabász — When to Use

✅ Best Scenarios

- **Large datasets** — $O(n)$ complexity beats Silhouette's $O(n^2)$
- **Quick model selection** — fast sweep over many k values
- **Compact, spherical clusters** — CH is most reliable here
- **Initial screening** — use CH first, then validate with Silhouette

❌ Limitations

- **Biased toward fewer, larger clusters** — tends to prefer $k = 2$
  - **Penalizes many small clusters** — even when they're genuinely compact
- **Assumes spherical clusters** — unreliable for non-spherical spherical
- **Sensitive to outliers** — extreme points inflate WCSS, reducing score

Calinski-Harabász — Usage Example & Interpretation

```python
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import calinski_harabasz_score

# 1. Generate data
X, _ = make_blobs(n_samples=300, centers=3,
                  cluster_std=0.8, random_state=42)

# 2. Evaluate k values — HIGHER CH is better
results = {}
for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42)
    labels = km.fit_predict(X)
    score = calinski_harabasz_score(X, labels)
    results[k] = score
    print(f"  k={k} → CH Score = {score:.2f}")

best_k = max(results, key=results.get)    # ← NOTE: max for CH
print(f"\n✅ Best k = {best_k}  (CH = {results[best_k]:.2f})")
```

```
  k=2 → CH Score = 892.14
  k=3 → CH Score = 1547.33   ← best (highest)
  k=4 → CH Score = 1203.67
  k=5 → CH Score = 981.45
  k=6 → CH Score = 834.22

✅ Best k = 3  (CH = 1547.33)
```

Interpretation: CH peaks at $k = 3$. High absolute values indicate dense, well-separated clusters.

⚖️ When to Use Which Metric

Decision Guide

```
Start
  │
  ├─ Large dataset (n > 50k)?
  │     ├─ YES → Use Calinski-Harabász (fast) or DBI (moderate)
  │     └─ NO  → Any metric works; Silhouette most informative
  │
  ├─ Need per-point diagnostics?
  │     └─ YES → Silhouette (use silhouette_samples)
  │
  ├─ Algorithm uses centroids (K-Means)?
  │     ├─ YES → DBI or CH preferred
  │     └─ NO (DBSCAN, hierarchical) → Silhouette more appropriate
  │
  └─ Final recommendation?
        → Use ALL THREE and look for CONSENSUS
```

💡 **Best practice:** Report all three; they often agree on the optimal k but provide complementary insights when they disagree.

🛠️ Quick Reference: `scikit-learn` API

```python
from sklearn.metrics import (
    silhouette_score,        # mean Silhouette → float
    silhouette_samples,      # per-point Silhouette → array(n,)
    davies_bouldin_score,    # DBI → float
    calinski_harabasz_score # CH  → float
)
# All three share the same signature:
score = metric_function(X, labels)
# X      : dataset, shape (n_samples, n_features)
# labels : cluster assignments


# e.g., silhouette_score also accepts a precomputed distance matrix:
score = silhouette_score(X, labels, metric='euclidean')  # default
```

Tip: For large datasets, use `silhouette_score(X, labels, sample_size=5000)` to compute on a random subsample — much faster, slightly approximate.

📜 Golden Rules

1. 📌 **No single metric is universally best** — use all three
2. 📌 **Internal metrics ≠ business value** — always validate with domain experts
3. 📌 **None work well for non-convex clusters** — visualize first!
4. 📌 **Consensus across metrics builds confidence** — they should agree at the true k